*For any suggestions or feedback regarding these notes, please contact [Pragy Agarwal]*

# The Update Problem

## SQL

### Strong & Static Schema

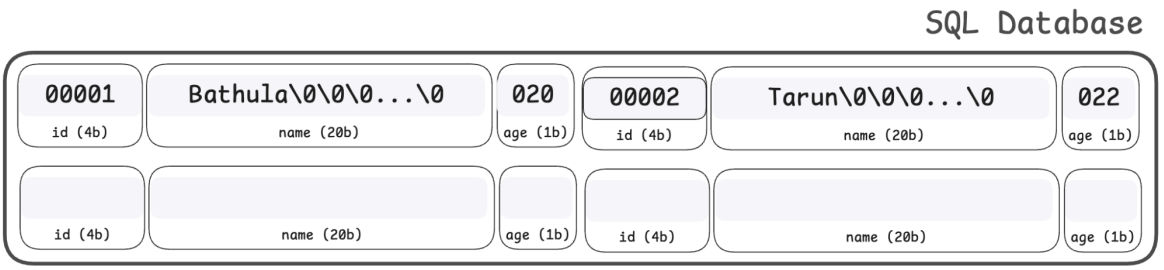
We know exactly what tables exist, what columns exist inside each table, what's the data type of each column.
Data type ⇒ size of the column.

We know what max size of any row inside a table will be (all the rows inside a given table have the same size, because they've the same set of columns).

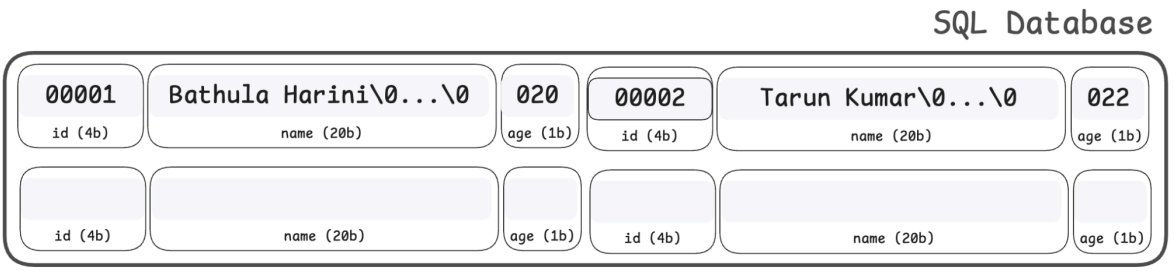
### Preallocating Space

Whenever SQL writes a row to the disk, it pre-allocates the **max space** required for each column. If the actual value is smaller than the max space, then the value will get **null padded**

```
users: id (4b), name (20b), age(1b) => 25b
```




### Updates



Updates in SQL are easy, because we've pre-allocated space already. The data will not overflow the pre-allocated space, so it will not overflow into adjacent values.

### What happens if the value is larger than the column's allocated space?

SQL will simply **truncate** the value, or **throw an error (🔗postgres docs)**
Schema tells us what the max allowed size is.
*Our assumption is, that since the developer created the schema, they're purposefully enforcing these size constraints – so it is okay to complain when these constraints are violated*

### How does adding/deleting a new column in a table work in SQL?

Since I've already packed the data one after the other on the disk, there's no space to add new column without overwriting the adjacent columns/rows. So there's no way to seamlessly add/delete columns inside a SQL table.

This causes an entire table rewrite. Very very slow!
(Database Migration)

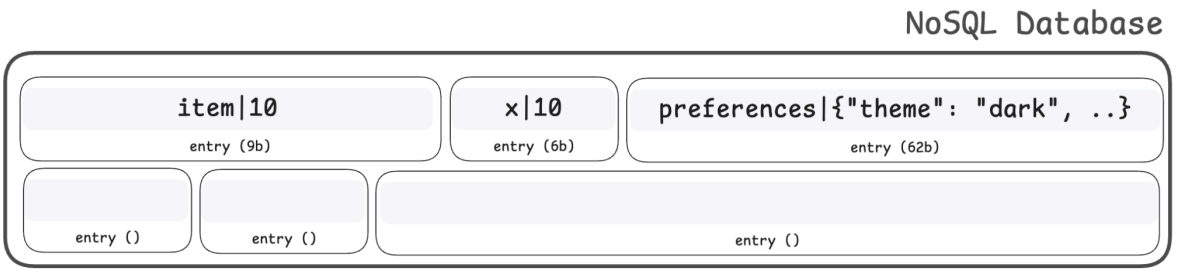
# NoSQL

NoSQL databases are mostly schemaless (or semi-structured / loose schema)
We don't know the size of a particular entry.

| Key | Value | Entry Size |
|---|---|---|
| item | 10 | 9 bytes<br>(key: 4b, value: 4b, separator: 1b) |
| preferences | `{`<br>`  "theme": "dark",`<br>`  "autoSave": false`<br>`}` | 62 bytes<br>(key: 11b, value: 50b, separator: 1b) |
| | | |

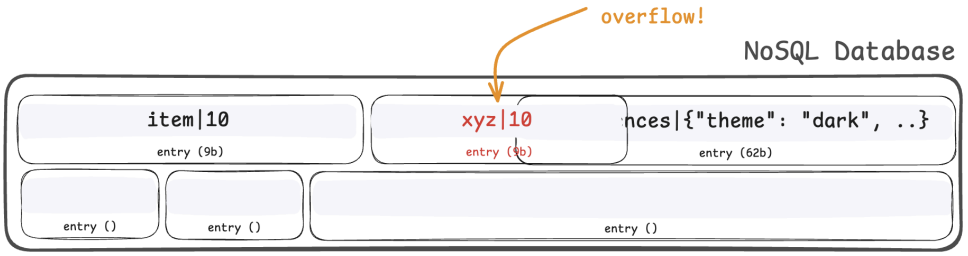
### No Preallocation

NoSQL databases do not pre-allocate max-space for an entry.
They canNOT pre-allocate max-space. Because the possible maximum is just too large (redis: size limit for string is 500MB) => preallocating such large values will be a massive waste of space.




### Updates

When we're updating an entry in NoSQL, then the size of the entry can change.




### What happens if the value is larger than the allocated space?

Updating a value when the size has increased will cause overflow - it will end up overwriting the adjacent entry.

1. ~~**Truncate:** enforce that you can't update a value to a larger size: bad design - useless database~~
2. ~~**Shift:** all the subsequent entries to the right - ridiculously slow~~

### Why is it okay to truncate large values in SQL but not in NoSQL?

In SQL, the developer decides the schema - the dev is aware & wants to enforce the max size. Truncation in SQL is not unexpected behavior.

In NoSQL the dev doesn't have any such schema. Truncation will be unexpected.

**Therefore, in NoSQL database, it is impossible to update the value on the disk in the traditional manner!**
Any entry inside a NoSQL database can only be appended - entries are immutable.
Challenge: how do you perform updates & deletes?

# Log-Structured Merge (LSM) tree

**Visualization:**
**https://www.pragy.dev/blog/lsm-trees-visualized**

# **Quick Persistence** - WAL file

Any operation (insert/update/delete) should be durable - persisted on the disk.

**Write-Ahead Log** (WAL) file is an append-only file on the hard-disk. Any new write *(insertion/deletion/updation)* to the database is just appended as a new entry at the end of the WAL file.
Because the file is append-only, the writes are sequential. The write throughput is high.

WAL file acts as temporary storage. **Data is committed & durable**, but it has not yet been fully absorbed into the database (internal bookkeeping is pending).

WAL file has a max size (typically: 100MB)
*Once the WAL file reaches max size ⇒ we must dump it into an SSTable.*

**Write Ahead Log (WAL)**

| | Key | Value | Time |
|---|---|---|---|
| 0 | b | 2 | 6 |
| 12 | c | 3 | 7 |
| 24 | d | 4 | 8 |

### Can we directly read from the WAL file?

Bad - WAL file is append only
- WAL file has duplicates
- WAL file is not sorted
- WAL file is large: 100MB

You will have to scan the entire 100MB to find the latest entry for the key.

### Can we have some sort of an index to give us quick access to the data?

Yes, MemTable!

# **Read Cache** - MemTable

MemTable is just a hashmap in the RAM.
*(a lot of times, MemTable is also implemented as a BBST Tree or a sorted linked list in RAM)*

While the writes must mandatorily go to the disk (durability), the reads can be served from the RAM (for ultra-high throughput).
We will maintain an in-memory hashmap => MemTable

MemTable acts as an in-memory cache.

- **Invalidation:** write-through cache, but it is super simple (no 2PC) because the entire LSM tree is within a single server.
- **Eviction:** Least Recently Used (LRU) eviction

**MemTable**

| Key | Value | Time |
|---|---|---|
| item | 1000 | 5 |
| point | {"x":5,"y":7} | 3 |
| a | 1 | 4 |
| b | 2 | 6 |
| c | 3 | 7 |
| d | 4 | 8 |

The maximum size of the MemTable will be limited by the server's RAM.
The more RAM we have, the larger the db-internal cache.

*Typically, the size of the MemTable is kept larger than the max size of the WAL file (simplifies our eviction & read queries)*

# **Long Term Persistence** - SSTables

SSTables are also files on the hard disk.
**Sorted String** (SS) Table: when the WAL file gets full, we dump it into a new SSTable.

The entries inside the SSTable are **sorted** by the key.
Entries inside the SSTable have no duplicates (we deduplicate the WAL file data before dumping in the SSTable)

**SSTable 0_1**

Sparse Index: *a,point*

| | Key | Value | Time |
|---|---|---|---|
| 0 | a | 1 | 4 |
| 12 | greeting | "Hello" | 1 |
| 37 | item | 1000 | 5 |
| 55 | point | {"x":5,"y":7} | 3 |

### Q: Can there be duplicate entries in the SSTables?

A single SSTable is deduplicated and sorted by key.
However, it is possible to have duplicate entries for the same key across different SSTables.

**SSTable 0**

Sparse Index: *greeting*

| | Key | Value | Time |
|---|---|---|---|
| 0 | greeting | "Hello" | 1 |
| 25 | item | 100 | 2 |

**SSTable 1**

Sparse Index: *a*

| | Key | Value | Time |
|---|---|---|---|
| 0 | a | 1 | 4 |
| 12 | item | 1000 | 5 |
| 30 | point | {"x":5,"y":7} | 3 |

Any old entries of the key will be "overridden" by the latest write.
Old entries of the key are now **redundant.**

# **Removing Redundancies** - Compaction

Whenever we've multiple SSTables, there's a possibility of having duplicate (redundant) entries across them, which leads to space wastage.

We therefore need to remove the duplicates.

Additionally, for reads, we have to scan the SSTables one by one, so we don't want the number of SSTables to be large. Therefore, we need compaction to reduce the number of SSTables.

**We will take 2 consecutive SSTables and we will merge them into a single SSTable.**
While merging, note that we can use the Merge algorithm from merge sort (because each SSTable is individually sorted by key). However, for duplicate key entries, instead of taking both entries, we will just take the latest entry.

**Note that after compaction the original SSTables get deleted.**

SSTable 0

Sparse Index: *greeting*

| | Key | Value | Time |
|---|---|---|---|
| 2 | greeting | "Hello" | 1 |
| 7 | item | 100 | 2 |

SSTable 1

Sparse Index: *a*

| | Key | Value | Time |
|---|---|---|---|
| 0 | a | 1 | 4 |
| 37 | item | 1000 | 5 |
| 55 | point | {"x":5,"y":7} | 3 |

SSTable 0_1

Sparse Index: *a,point*

| | Key | Value | Time |
|---|---|---|---|
| 0 | a | 1 | 4 |
| 12 | greeting | "Hello" | 1 |
| 37 | item | 1000 | 5 |
| 55 | point | {"x":5,"y":7} | 3 |

## Q: When does compaction happen?

Compaction requires ***at least 2 tables*** on the same level.
If there's 2 tables on the same level, we can compact them to the next level.

Compacting happens in the background during the usual database operations. There's no downtime.

***However, compaction is expensive (reads, writes, deletes on disk)***

Therefore, we don't usually do compaction immediately when there's 2 tables on the same level.

Compaction can be done according to
1. amount of tables (tableCountThreshold)
2. time passed (compact at every midnight)
3. low load time (when the load is minimum or below a threshold, detect it, and start compacting)
4. ...

**Most Common Compaction Strategies**
1. **Levelling:** the moment you get 2 SSTables on a level, you trigger compaction. `tableCountThreshold = 2`
2. **Tiering:** there can be more than 2 SSTables on a level - you maintain a compaction threshold `tableCountThreshold > 2`

The compaction strategy is decided differently for each level.

Poorly tuned compaction can make or break the database performance. You've to be careful while choosing the compaction strategy.

Note that you don't have to read the entire file into RAM while compacting, you can do it in a streaming manner (because the tables are sorted).

### Resources (optional)

- LSM-Tree Compaction Visualization
- An In-depth Discussion on the LSM Compaction Mechanism - Alibaba Cloud Community
- Raphael Carvalho — Building state-of-art LSM-tree compaction in ScyllaDB

# Optimizing Disk Reads - Sparse Indices

Binary Search `O(log n)` is ultra fast compared to Linear Search `O(n)`
Consider a database with 1 trillion entries.
Binary Search takes $O(\log_2 1 \text{ trillion}) = 40$ steps
Linear Search takes = 1 trillion steps
***Binary Search is 25 billion times faster than Linear Search in this example!***

Binary Searching the SSTable is expensive, because the SSTable size can be large.

Typical WAL size `= 100MB`

size SSTable on Level 1 `<= 100MB` (because it is compacted from the WAL file)

size SSTable on Level 2 `<= 200MB` (because it is compacted from 2 SSTables from Level 1)

size SSTable on Level 3 `<= 400MB` (because it is compacted from 2 SSTables from Level 2)
...

Consider an SSTable of size `100GB`. Assume each entry (key, value, timestamp) on average is `100 bytes`.
Number of entries in the SSTable `= 100G bytes / 100 bytes = 1G = 1 billion`
For a binary search, # iterations = $\log_2 (10^9) = 9 * \log_2(10) \approx 30$ iterations
Each iteration on the disk is a random read. Each random read takes `~18ms`
Total time for checking 1 SSTable `= 540 ms`

If we have 10 SSTables, then each read potentially takes `5.4 seconds` worst case.
Therefore, we need an index.

### Idea

What if we get rid of the binary search O(log n) and instead use an in-memory (RAM) index to exactly pin-point the location of an entry in the SSTable O(1).
This will give us a 30x speedup (in the above example of SSTable with 1 billion entries)

So each SSTable lookup is now just 18 ms, and since you've 10 SSTables, your reads take `180ms` total worst case (30x faster compared to 5.4 seconds).

Notice that this is not always the case — 90% of the reads are being served directly from the RAM thanks to the MemTable which acts as a read cache. Reading from the MemTable takes `0.1ms` at worst.
Effective read latency is actually much better. Only 10% of you times you need to hit the disk, and in the worst case of that, you will hit `180ms` of read latency
Effective read latency `~20ms` on average.

## Full Index

Instead of binary searching the SSTable, what if we maintain a hashmap of `{key: offset}` in the RAM for each SSTable.
Given a key, we don't have to binary search the SSTable.
We just check the Index to find if the key is there in the SSTable. If the key is there in the index, the index will give us the exact offset within the file to read.
Reads: `O(1)` per SSTable. **`30x` speedup**

However maintaining a full index in the RAM will take a lot of space.
Our SSTable has 1 billion entries. So the index will also have 1 billion entries.
That's too much RAM.

## Sparse Index

Sparse Index is built for each SSTable, and is stored in the RAM.
Sparse Index is built when the SSTable is created.

Because data is always read/written in blocks, we don't need to maintain an index of all the keys in the SSTables.
We can just maintain an index of the first key of each block.
Typical block size: `4KB`
Assume each entry (key, value, timestamp) on average is `100 bytes`.
Keys in each block `4K bytes / 100 bytes = 40 keys`

Our sparse index will be smaller in size by a factor of 40.
Sparse Index will now only have 25 million entries `(~100MB)` for a 100GB SSTable.

Sparse Index is just a sorted list of keys (we only take the first key of each block) of the SSTable. Sparse Index is stored in the RAM.

# Deletion - Tombstones

### ~~Naive Approach 1~~

1. Delete the key from MemTable

Will this work? NO!

After this delete, if we try to read the value, we will end up searching the SSTables.
Effectively, this approach will not really delete the key, it will just undo the last key.

### ~~Naive Approach 2~~

1. Delete the key from Memtable
2. Delete the key from WAL
3. Delete the key from all SSTables

This will certainly get rid of the key.
Incredibly expensive.
Note that SSTables are immutable. If you delete data, since you can't shift all the other data, there will be gaps in the disk

**Tombstone** (aka Sentinel / Flag / Marker / ..) **is a special marker that indicates that the value has been deleted.**

Any deletes happen via writes.

```
void set(key, value) {
    ...
}
void delete(key) {
    set(key, TOMBSTONE)
}

string _get(key) {
    // check memtable
    // check sstables
    // ...
}

string get(key) {
    value = _get(key)
    if (value == TOMBSTONE) {
        raise KeyNotFoundError!
    }
    return value
}
```

**Note: Compaction can only delete Tombstones from the oldest SSTable (the one starting from SSTable 0)**
If an SSTable is not the oldest, the tombstone entries must still be stored in the SSTable.

# Final Structure & Algorithm

## Writes

1. We first append the entry into the WAL file (for durability)
2. We then set the value in the Memtable (for fast reads)

The following events can be triggered after any write:

- **Flush:** If the WAL file gets full, we flush it into a new SSTable.
- **Compaction:** If there's too many SSTables on any level, we trigger compaction.
- **Eviction:** If the MemTable gets full, we use LRU eviction to remove the oldest entry.

*Deletions happen by setting the value to TOMBSTONE*
*Bloom Filter also gets updated for each write*

## Reads

1. **We first check the MemTable** (which acts as a cache)**.**

a. If the data is found in the MemTable, then the value is guaranteed to the latest value
    i. because any changes first go to the MemTable (write-through) + WAL
    ii. data in MemTable can never be stale
b. Any recently written/accessed data is guaranteed to be in the MemTable
c. Majority of the reads are incredibly fast — `O(1)` access in the RAM (if your cache hit ratio is high)
d. Majority of the time, we won't even have to touch the disk
e. Note that after reading the value, we must update the last accessed timestamp in the MemTable for that key.

**2. If we don't find the data in the MemTable**

a. Note that we don't have to check the WAL file
    i. because the MemTable is significantly larger than WAL and follows LRU eviction, it means that any data in the WAL will also be in the MemTable
b. We must now scan **ALL** the SSTables one by one, in reverse order *(starting from the most recent to the least recent)*.

    **For each SSTable, do:**

    i. Check if the key is present in the SSTable
        1. ~~since the SSTable is sorted, this will `O(log n)` disk reads (`n` is the number of entries in the SSTable)~~
        2. First, Binary Search (lowerbound) the key in the Sparse Index from the RAM for this SSTable.
           The SparseIndex will tell us which block of the SSTable to read.
           *this requires no disk reads, since the Sparse Index is in the RAM*
        3. We will read the entire block into the RAM
           *Reading the entire block takes just 1 disk access*
        4. We can now Binary Search this block within the RAM
           *RAM is 1 million times faster than disk ⇒ binary search in RAM is free*
        5. If the key is found, then this must be the latest value (because we're scanning from recent to old)
           We can return this value
        6. If not found, we continue to the next older SSTable.

ii. If the key is not found in any SSTable, then we raise a **KeyNotFoundError**

iii. Once we find the value, we must update the MemTable so that any subsequent accesses for this key are fast

Note: during a read, if the value is found to be TOMBSTONE, we return a **KeyNotFoundError**

### Isn't scanning all SSTables one by one a slow process?

Yes. This is why, we've the following optimizations

1. **Compaction:** will reduce the number of SSTables.
    - Typically database LSM tree will have `<= 10` SSTables
2. **Sparse Index:** Binary Search on disk is slow. Reduce that to O(1) search.

# Optimizing containment checks - Bloom Filter

If we try to `get(key)` for a key which was never inserted in the DB, then this leads to the slowest possible read - worst case for reads!

- first check MemTable - we won't find it (never inserted)
- scan each SSTable one by one - we won't fit it (never inserted)

In a lot of databases, it is common to have to check if an entry was never inserted.

*For example - signing up to a website - you have to create a username, this username must be distinct. The user will supply a unique username `pragy1234` - we must ensure that this username doesn't exist in the DB already. This is an example of a read for a key that was never inserted.*

**Bloom Filter**

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | | 1 | 1 | | 1 | | 1 | | | | 1 | | | 1 | 1 | 1 | 1 | | 1 | 1 | 1 | | 1 |

- Bloom Filter is a **probabilistic Data Structure**
- Bloom Filters only support **Inserts & Containment Checks**.
  They do NOT support reads / updates / deletes / iteration
- Containment checks can be faulty (it can lie to you)

    - **False Positive is possible:** If a key was not inserted in the bloom filter, it can still say that the key exists
    - **False Negative is impossible:** If a key was inserted in the bloom filter, it will NEVER say that the key does not exist.

BloomFilter is just a bit array.

- Whenever we insert anything into our LSMTree, we will also insert the key into the bloom filter.
- Whenever we read a value from the LSMTree, we will first check if the key exists in the Bloom Filter
    - Bloom Filter says "Yes, key exists" - we will scan the SSTables
        - it is possible for the key to be actually present in the database
        - it is also possible for the key to not be present in the database (bloom filter gave a false positive)
    - Bloom Filter says "No, key not found":
        - guaranteed that this key was never inserted in the DB. Just return KeyNotFound!

```
void set(key, value) {
    bloom_filter.insert(key)
    // ...
}

string get(key) {
    if(! bloom_filter.contains(key))
        raise KeyNotFound!
    // ...
}
```

### **Resources** (optional)

- Quick Demo: https://llimllib.github.io/bloomfilter-tutorial/
- Math & Code: https://brilliant.org/wiki/bloom-filter/
- Explanation:
    - How to Use Bloom Filters in Redis
    - Bloom Filters | Algorithms You Should Know #2 | Real-world Examples
    - Bloom Filters Explained by Example
    - What are Probabilistic Data Structures: Bloom Filters
- Calculate optimal capacity & hashes: https://hur.st/bloomfilter/

# Extending to other Databases

## Document DB

## Column Family DB

# WAL file in SQL

WAL files are used in NoSQL dbs as a part of the LSM tree.
But, WAL files are also used extensively in SQL databases.

## 1. Write Performance - Batching

A SQL database has lots of things that are going on. Data needs to be written inside the tables (contiguous rows on the disk). Also, we need to keep the B+ tree indexes updated.

If you want to insert/update/delete something in a SQL database, that potentially causes a lot of random reads/writes.

1. you need to update the row (1 random read + 1 random write)
2. you need to update all the B+ tree indexes for this table (say there's 3 indexes on this table, we need to update all 3)
    a. for each index: we need to go to the correct node (O(log n) random reads) and then update that node (1 random write)
    b. we have to potentially re-balance the tree this can cause another O(log n) random reads and O(log n) random writes
3. A single transaction might touch multiple rows — so we need to do the above process for each row

In the worst case, a single transaction might incur 20-30 random reads & writes — incredibly slow (`3-5 seconds`)

Therefore, WAL file to the rescue! Instead of doing all this, we simply append the changes to the WAL file —- this is just a sequential write (this only takes `10-20 ms` — almost 300x times faster!)

Now of course the bookkeeping needs to be done. Once the WAL file becomes full (100MB), then we will do the rest of the stuff (updating the rows, updating the B+tree

indexes) can be done later as a batch. Because we're doing it as a batch, we can do optimizations (better disk seeking algos that will incur less random i/o)

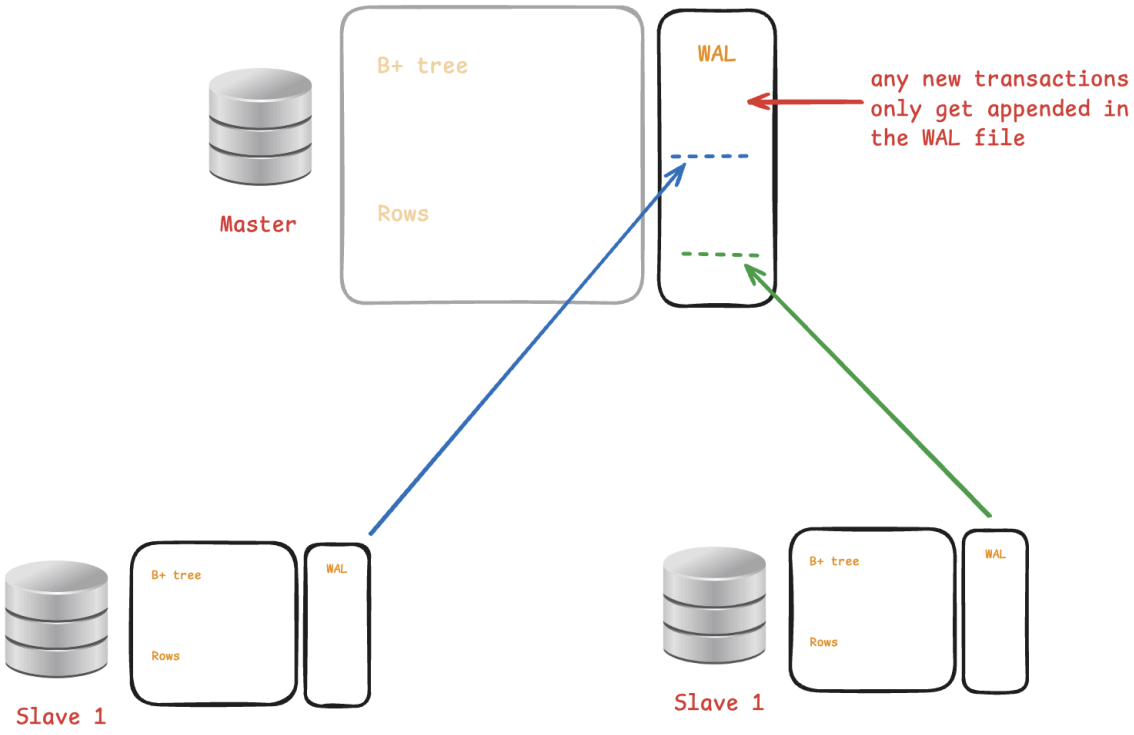
## 2. Master-Slave Replication

Running read queries on the master for syncing up the slaves will be expensive — it will cause the database to slow down.

```
select *
from all_tables
where modified_at > (now() - 1 hour)
```

So instead of that, for slave sync-up, the slave simply maintains a "offset" in the master's WAL file.

For the sync up, it just requests the new data from the WAL file starting from its offset.

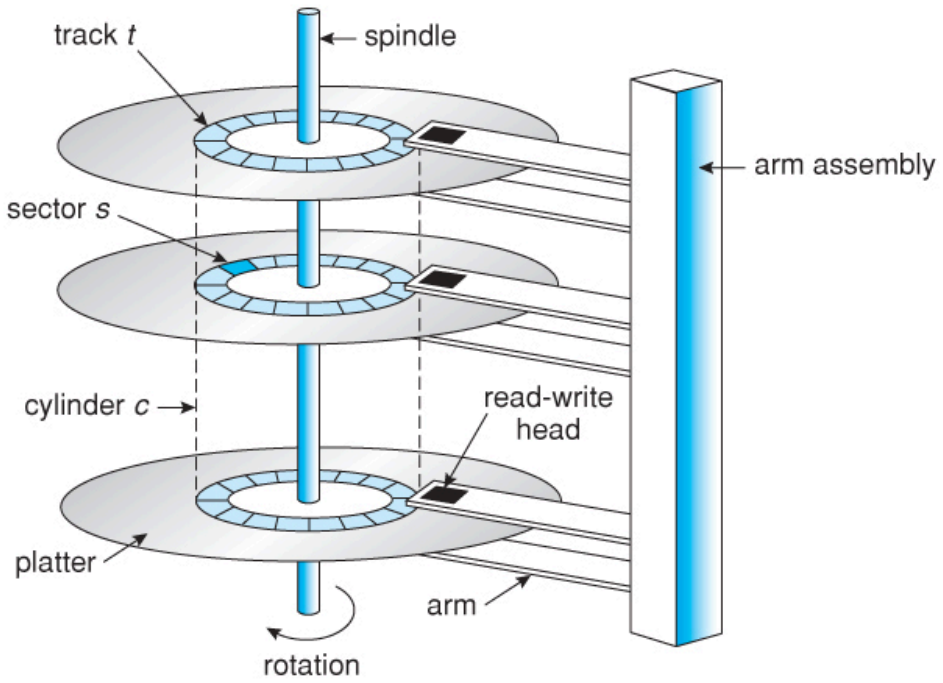


# How Storage Works

1. **Magnetic:** https://www.youtube.com/watch?v=wtdnatmVdIg
2. **Solid State:** https://www.youtube.com/watch?v=5Mh3o886qpg
3. **RAM:** https://www.youtube.com/watch?v=7J7X7aZvMXQ

Modern Magnetic Disks rotate at ~7200 RPM (120 rotations per second, or 8ms per rotation)

- **Sequential Read/Write:** If you want to read sequential data (data within the same ring/track), it is very fast.
  Just move the read head to the correct ring (10ms), and spin the disk (8ms).
  Within 18ms, you will be able to read 10s of MBs of sequential data.
  Throughput: 500 MBps
- **Random Read/Write:** For reading just 1 byte from a random location, you have to move the read head there (10ms), spin the disk (8ms).
  Within 18ms you will be able to read 1 byte of random data.
  Throughput: 55 bytes / second



# Sequential vs Random Access

Sequential access is up to 10,000x - 100,000x faster than random access!
*True irrespective of the storage technology (HDD, SSD, RAM, L3 cache, Brain)*

# Data on a disk is always read in blocks!

Reading 1 byte of data at location X:

- move the read head (10ms)
- spin the disk (8ms)

While you're spinning the disk, can't you just read the entire ring (or at least the part that you've spun through)?

Basically, if you want to read 1 byte of data from the disk, you get all the other bytes of that sector for free!

Typical Disk Sector Size: 512 bytes - 4 KB

**Therefore, you always read the data in blocks.**